

How a Transformer Actually Works — A Tiny Worked Example

Everyone talks about transformers. Very few explanations show you the actual arithmetic. So let's do that — with numbers small enough to check on paper.

The problem transformers solve

Take the word "**bank**." On its own it's ambiguous — a place for money, or the edge of a river? The only way to know is to look at the words around it.

Older models (RNNs, LSTMs) read words one at a time, left to right, and struggled to carry context across long distances. The transformer's insight, from the 2017 paper *Attention Is All You Need*, was simple: **let every word look at every other word simultaneously, and decide for itself which ones matter.**

That mechanism is called **self-attention**.

The example

Our sentence is two words: "**river bank**"

Real models use vectors with hundreds or thousands of dimensions. We'll use two, so the math stays readable.

Step 1 — Words become vectors (embeddings)

```
river = [1, 0]
bank  = [0, 1]
```

Step 2 — Each word produces three things: Query, Key, Value

Think of it as a search engine inside the sentence:

- **Query (Q)** — "here's what I'm looking for"
- **Key (K)** — "here's what I offer"
- **Value (V)** — "here's the information I'll actually hand over"

Each is made by multiplying the embedding by a learned weight matrix. These matrices are what the model learns during training. Ours are made up:

```
W_Q = [[0, 1], [1, 0]]
W_K = [[1, 0], [0, 1]]
W_V = [[2, 0], [0, 1]]
```

Focus on the word "**bank**" and find what it should pay attention to.

```
q_bank = [0, 1] × W_Q = [1, 0]
k_river = [1, 0] × W_K = [1, 0]
k_bank  = [0, 1] × W_K = [0, 1]
```

Step 3 — Score every word against the query (dot product)

```
bank → river:  [1,0] · [1,0] = 1
bank → bank :  [1,0] · [0,1] = 0
```

"bank" finds a strong match with "river" and none with itself.

Step 4 — Scale and softmax

Scores are divided by $\sqrt{d}$ (here $\sqrt{2} \approx 1.41$) to keep gradients stable, then passed through softmax to turn them into weights that sum to 1.

```
scaled:  [0.71, 0]
softmax: [0.67, 0.33]
```

These are the attention weights. "bank" will draw 67% of its new meaning from "river" and 33% from itself.

Step 5 — Blend the Values

```
v_river = [1, 0] × W_V = [2, 0]
v_bank  = [0, 1] × W_V = [0, 1]

output = 0.67 × [2, 0] + 0.33 × [0, 1]
        = [1.34, 0.33]
```

The word "bank" started as $[0, 1]$. It leaves as $[1.34, 0.33]$ — now dominated by the "river" direction.

That's it. That's attention. The word has been rewritten in terms of its context. No recurrence, no loops — just matrix multiplication, which GPUs happen to be extremely good at.

What the real thing adds

The mechanism above is the whole idea. Production transformers scale it in four ways:

1. **Multi-head attention** — run 8-96 of these in parallel. One head tracks grammar, another tracks subject-object links, another tracks tone. Their outputs get concatenated.
2. **Feed-forward layers** — a small neural network applied to each position afterwards, where much of the model's factual knowledge is thought to live.
3. **Positional encodings** — attention alone is order-blind ("dog bites man" = "man bites dog"), so position information is added to every embedding.
4. **Stacking** — repeat the block 12, 96, or more times. Early layers catch syntax; later layers catch meaning and intent.

Residual connections and layer normalisation wrap each block to keep training stable at that depth.

Why this matters

Because attention has no sequential dependency, every token is processed at once. That parallelism is what made training on internet-scale data affordable — and it's the reason a 2017 translation architecture now underpins ChatGPT, Claude, protein folding, and image generation alike.

Attention is the piece that makes the whole architecture possible. Understand it, and the rest of a modern language model is largely a matter of scale, repetition, and plumbing.

Short version for LinkedIn

Everyone uses transformers. Few can explain the actual math. Here it is in 60 seconds.

Take the phrase "river bank." How does a model know we're not talking about money? Each word generates three vectors: a **Query** (what I'm looking for), a **Key** (what I offer), and a **Value** (what I'll contribute).

"bank" sends out its Query. It compares against every Key in the sentence via a dot product. Match with "river": 1. Match with itself: 0.

Softmax turns those scores into weights: **67% river, 33% bank.**

The model then blends the Values in that ratio. "bank" is literally rewritten as a mixture of its context. The financial meaning never enters the picture.

Multiply that by 96 attention heads across 96 stacked layers, and you have GPT.

No recurrence. No sequential bottleneck. Just matrix multiplication — which is exactly why GPUs made this era possible.

The full walkthrough with the arithmetic is on my blog 📌

#MachineLearning #DeepLearning #AI #Transformers #LLM